Answer Script Evaluation Using OCR, Semantic Analysis and Generative AI

PHASE I REPORT

Submitted By

SIVASABARI GANESAN A	**230701321**
GOKULAKRISHNAN K	**230701094**
GIRIDHARAN E	**230701091**
HARI SANKAR S	**230701100**

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

DECEMBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**ANSWER SCRIPT EVALUATION USING OCR, SEMANTIC ANALYSIS AND GENERATIVE AI**" is the bonafide work of **SIVASABARI GANESAN A (230701321), GOKULAKRISHNAN K (230701094), GIRIDHARAN E (230701091), HARI SANKAR S (230701100)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

This project addresses the following Sustainable Development Goal(s):

SDG Goal No: 4 – Quality Education – Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SIGNATURE

Dr. J. Manoranjini, Ph.D.,
Professor and Head,
Department of Computer Science & Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Ms. Jaeyalakshmi M, M.E.,
Assistant Professor (SS),
Department of Computer Science & Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Handwritten and scanned answer-script evaluation is time-consuming and may produce inconsistent marking due to evaluator fatigue and subjectivity. Traditional keyword and string-similarity methods also fail to recognize correct answers expressed differently.

This project presents an AI-Based Answer Script Evaluator, a human-in-the-loop system that assists faculty in evaluating scanned scripts and PDFs. PaddleOCR, with ONNX-runtime RapidOCR fallback, extracts handwritten and printed text. all-MiniLM-L6-v2 generates semantic embeddings, while Qdrant retrieves relevant reference content, answer keys, and rubric points. Multiple LLM agents perform evaluation using Cerebras Cloud API, NVIDIA NIM, and Groq.

Answers are flagged for teacher review when confidence is below 0.80 or agent disagreement exceeds 1.5 marks. The system achieved 1.42 s/page OCR, 18.5 ms/question retrieval, 2.15 s/page evaluation, and Pearson correlation $r = 0.94$ with expert faculty marking. Overall, it reduces manual effort, improves marking consistency, and provides actionable feedback while keeping faculty in control.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report.

Our sincere thanks to our Chairman **Mr. S. MEGANATHAN**, B.E., F.I.E., our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN**, B.E., M.S., and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN**, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to **Dr. J. Manoranjini**, Ph.D., Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Ms. Jaeyalakshmi M**, M.E., Assistant Professor (SS), Department of Computer Science and Engineering for her valuable guidance throughout the course of the project.

We thank our Project Coordinator, **Dr. S. MADHUSUDHANAN**, Ph.D., Associate Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

SIVASABARI GANESAN A	**:**	**230701321**
GOKULAKRISHNAN K	**:**	**230701094**
GIRIDHARAN E	**:**	**230701091**
HARI SANKAR S	**:**	**230701100**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**ix**
	LIST OF FIGURES	**x**
	LIST OF ABBREVIATIONS	**xi**
1.	**INTRODUCTION**	**1**
	1.1 GENERAL	1
	1.2 MOTIVATION AND SCOPE	2
	1.3 OBJECTIVE	2
	1.4 EXISTING SYSTEM	3
	1.4.1 LIMITATIONS OF EXISTING SYSTEM	4
	1.5 PROPOSED SYSTEM	4
	1.6 ORGANIZATION OF THE REPORT	5
	1.7 PROBLEM STATEMENT	5
	1.8 ADVANTAGES OF THE PROPOSED SYSTEM	6
	1.9 KEY CONTRIBUTIONS OF THE PROJECT	6
2.	**LITERATURE SURVEY**	**8**
	2.1 LITERATURE REVIEW	8
	2.2 COMPARISON AND DISCUSSION	10
	2.3 RESEARCH GAP AND SUMMARY	11
	2.4 CONCLUSION	11
	2.5 TECHNOLOGY BACKGROUND	11
	2.5.1 OPTICAL CHARACTER RECOGNITION	11
	2.5.2 SENTENCE EMBEDDINGS AND SEMANTIC SIMILARITY	12
	2.5.3 VECTOR DATABASES	12

2.5.4 LARGE LANGUAGE MODELS AND AGENTS 12
2.5.5 RETRIEVAL-AUGMENTED GENERATION 12
2.5.6 HUMAN-IN-THE-LOOP ASSESSMENT 13
3. SYSTEM DESIGN 14
3.1 SYSTEM ARCHITECTURE 14
3.2 SYSTEM REQUIREMENTS 15
3.2.1 SOFTWARE REQUIREMENTS 15
3.2.2 HARDWARE REQUIREMENTS 16
3.2.3 DATASET AND REFERENCE REQUIREMENTS 16
3.2.4 DEPLOYMENT AND SCALING ARCHITECTURE 17
3.3 SYSTEM DIAGRAMS 17
3.3.1 ACTIVITY DIAGRAM 17
3.3.2 DATAFLOW DIAGRAM 18
3.4 DATABASE DESIGN 19
3.5 PROCESSING WORKFLOW AND JOB QUEUE 20
3.6 SECURITY AND ACCESS CONTROL 21
3.7 DESIGN CONSIDERATIONS 21
3.8 SUMMARY OF THE DESIGN 22
4. PROJECT DESCRIPTION AND METHODOLOGIES 23
4.1 METHODOLOGIES 23
4.1.1 PROBLEM FORMULATION AND MATHEMATICAL MODEL 23
4.1.2 DATA PREPROCESSING AND OCR PIPELINE 23
4.1.3 QUESTION AND ANSWER SEGMENTATION 24
4.1.4 VECTOR STORE RETRIEVAL AND INDEXING 24

4.1.5 MULTI-AGENT EVALUATION AND SECONDARY VERIFICATION 24
4.1.6 WORKED EXAMPLE OF THE SCORING MODEL 25
4.2 SYSTEM MODULES 25
4.2.1 INPUT DATA PROCESSING MODULE 25
4.2.2 OCR AND ANSWER EXTRACTION MODULE 26
4.2.3 QUESTION AND ANSWER SEGMENTATION MODULE 27
4.2.4 AI-BASED ANSWER EVALUATION MODULE 27
4.2.5 FEEDBACK GENERATION MODULE 28
4.2.6 RESULT MANAGEMENT AND PRESENTATION MODULE 29
4.2.7 AUTHENTICATION SUPPORTING FEATURE 29
4.3 ALGORITHMS 30
4.3.1 ANSWER EVALUATION PROCEDURE 30
4.3.2 REVIEW ROUTING DECISION 30
4.4 AGENT ROLES AND PROMPT DESIGN 31
4.5 ERROR HANDLING AND PROVIDER FAILOVER 32
4.6 SUMMARY OF THE METHODOLOGY 32
4.7 WORKFLOW OF THE USER INTERFACE 33
5. IMPLEMENTATION AND RESULT DISCUSSION 34
5.1 IMPLEMENTATION AND RESULTS 34
5.1.1 AUTOMATED TEST SUITE RESULTS 34
5.1.2 SYSTEM PERFORMANCE AND LATENCY RESULTS 35
5.2 DISCUSSION AND BENCHMARK ANALYSIS 36
5.3 TESTING STRATEGY 36
5.4 LIMITATIONS 37

5.5 SUMMARY OF KEY RESULTS 38
5.6 OBSERVATIONS FROM THE SAMPLE EVALUATION RUN 38
5.7 PRACTICAL IMPLICATIONS FOR EVALUATORS 39
6. CONCLUSION AND FUTURE WORK 40
6.1 CONCLUSION 40
6.2 FUTURE WORK 40
6.3 SOCIAL RELEVANCE AND SDG ALIGNMENT 41
6.4 LEARNING OUTCOMES 41
6.5 PROPOSED PLAN FOR PHASE II 42
APPENDIX A – CODE AND DETAILS 43
A.1 SOFTWARE AND MODEL INVENTORY 43
A.2 SYSTEM THRESHOLDS AND DECISION LOGIC 44
A.3 CONCURRENCY CAPS AND RATE LIMITER 44
A.4 RETRIEVAL AND QDRANT PARAMETERS 45
A.5 MULTI-PROVIDER FAILOVER ORDER 45
A.6 AGENT PROMPT STRUCTURE AND JSON REPLY FORMAT 45
A.7 SOURCE CODE EXCERPTS 47
A.8 TEST SUITE INVENTORY 49
A.9 DATABASE SCHEMA 50
APPENDIX B – PUBLICATIONS 51
B.1 PUBLICATION STATUS SUMMARY 51
B.2 DETAILS OF EACH PUBLICATION 51
B.3 PROPOSED MANUSCRIPT OUTLINE 53
B.4 SUPPORTING DOCUMENTS 53
REFERENCES 54

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparison of Recent Short Answer Evaluation Studies	10
3.1	Software Requirements Specification	15
3.2	Hardware Requirements Specification	16
3.3	Core Database Entities	19
4.1	Review Routing Rules	30
4.2	Responsibilities of the Evaluation Agents	31
5.1	Automated Test Suite Executive Summary	34
5.2	Module-by-Module Test Suite Breakdown	35
5.3	Summary of Key Results	38
A.1	Software and Model Inventory	43
A.2	Provider Concurrency and Rate Limits	44
A.3	Retrieval and Qdrant Parameters	45
A.4	Test Suite Inventory	49
A.5	Database Schema	50
B.1	Publication Status Summary	51
B.2	Publication Details	51

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Answer Evaluation Processing Pipeline Architecture Diagram	14
3.3.1	System Processing and Multi-Agent Activity Diagram	18
3.3.2	System Data Flow Diagram (DFD Level 1)	19
4.1	AI Evaluator Main Dashboard and Workflow Pipeline	26
4.2	Examinations and Course Subject Management View	26
4.3	Question Paper Digitization and OCR Table Extraction (Part 1)	27
4.4	Question Paper Digitization and OCR Table Extraction (Part 2)	27
4.5	AI Evaluation Summary and Answer Comparison View	28
4.6	Step-by-Step Mark Allocation and Deduction Reasoning	28
4.7	Multi-Agent Deliberation and Automated Feedback Panel	29
4.8	Teacher Review and Score Overriding Interface	29
5.1	System Analytics and Evaluation Statistics Dashboard	36

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
DFD	Data Flow Diagram
ER	Entity Relationship
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
LLM	Large Language Model
MAE	Mean Absolute Error
NIM	NVIDIA Inference Microservice
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORM	Object Relational Mapper
PDF	Portable Document Format
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm 256-bit
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

CHAPTER 1
INTRODUCTION

1.1 GENERAL

Checking handwritten answer scripts is a demanding part of a faculty member's duties. After each mid-term or semester examination, the evaluator reads every student's answer, judges how clearly the concept has been explained, compares it with the official answer key and the marking rubric, and awards marks step by step. Repeated over hundreds of students and several subjects, this takes a great deal of time and concentration.

Manual marking also has human limits. Long sessions tire the evaluator, so the first scripts of a batch may be judged by slightly different standards than the last ones. When several faculty members share one paper, they may also give different credit to an answer that is only partly correct. Problems like these show why a systematic aid for evaluation is useful: it can help keep marking fair and consistent.

Early attempts at automatic grading relied on string-similarity measures, exact keyword counts or regular expressions. Such surface-level matching does not work well for open-ended short answers. A student may explain a correct idea using synonyms or a different sentence structure, and an answer key written in other words will not match it. Losing marks only because a keyword is missing is unfair to the student and weakens the evaluation itself.

The AI-Based Answer Script Evaluator is designed to close this gap. It combines Optical Character Recognition (OCR), dense vector embeddings, vector database retrieval and multi-agent Large Language Model (LLM) reasoning in one human-in-the-loop platform. The platform handles script upload, text extraction, question-answer segmentation, retrieval of reference context, scoring by several agents and feedback generation. The final decision is still left with

people, because low-confidence cases go through a second verification and teachers can override any mark.

1.2 MOTIVATION AND SCOPE

Student numbers in higher education keep growing, and evaluation has to scale without lowering academic standards. Automated Essay Scoring (AES) has matured for standardised tests, but short answers that need step-by-step reasoning have received much less attention. This project is motivated by the need for a practical, domain-aware evaluation engine that connects modern AI methods with the grading rules actually followed in institutions.

The scope covers the full evaluation workflow. It begins with uploading PDF or image answer scripts together with digitised question papers and rubrics, then extracts the text using PaddleOCR, retrieves textbook context from Qdrant, scores the answers with multiple LLM agents (Cerebras Cloud API, NVIDIA NIM and Groq), checks the confidence of each score, writes explanatory feedback for the student, and finally shows structured results on a dashboard for the evaluator.

1.3 OBJECTIVE

- To automate the intake, validation and pre-processing of scanned answer scripts submitted as images or PDFs.
- To read handwritten and printed text with PaddleOCR, keeping a RapidOCR ONNX fallback so that the system also runs on CPU-only machines.
- To split the extracted text into separate question-answer blocks that follow the official question paper and marking scheme.
- To turn academic reference material (textbooks, rubrics and answer keys) into 384-dimensional embeddings using the SentenceTransformers model all-MiniLM-L6-v2.

- To index these embeddings in a Qdrant collection named answer_chunks, with payload filtering by submission ID.
- To build a multi-agent LLM framework made up of AcademicContextEvaluator, WebRelevanceEvaluator, FeedbackAgent, OrchestratorHeadAgent and UnifiedExaminer.
- To award marks according to concept coverage and rubric criteria instead of keyword matching.
- To give students targeted feedback that points out what they explained well, what is missing and what to revise.
- To add an automatic verification step on NVIDIA NIM that flags evaluations with confidence below 0.80, or score variance above 1.5 marks, for teacher review.
- To offer evaluators a React and TypeScript interface where they can inspect AI predictions and agent discussion logs, and override marks.
- To build the back end in Python FastAPI, with Redis caching, job queueing and bounded concurrency (asyncio.Semaphore, maximum 5).
- To check the reliability of the system through an automated test suite covering OCR, authentication, job queues, multi-agent arbitration and multilingual translation.

1.4 EXISTING SYSTEM

In most universities today, answer scripts are still evaluated by hand. Booklets are collected, sorted by register number and handed over to the assigned evaluators. Each evaluator reads the script line by line, compares it with the printed marking key, writes the marks on a cover sheet or in a spreadsheet and adds up the total. When a course involves several departments, the marks have to be compiled manually, which leaves room for data-entry mistakes.

Where software is used at all, it is usually based on string-distance measures such as Jaro-Winkler or Levenshtein distance, or on simple regex keyword

counts. These tools have no sense of meaning. If a student explains a scientific or engineering concept in different vocabulary, the tool cannot recognise it as correct.

1.4.1 LIMITATIONS OF EXISTING SYSTEM

- **High manual effort:** Evaluators spend many hours reading scripts, which delays the declaration of results.
- **Inconsistency between evaluators:** Different faculty members apply their own judgement when giving partial marks to incomplete answers.
- **Fatigue and human error:** Accuracy drops over long marking sessions, which leads to more re-checking and re-evaluation requests.
- **Rigid keyword matching:** Keyword-based tools penalise correct answers that are written in different words.
- **Lack of transparency:** Students get only a number and no explanation of which concepts they missed.
- **Poor scalability:** Manual marking cannot keep up with the load at mid-term and semester examination peaks.

1.5 PROPOSED SYSTEM

The proposed AI-Based Answer Script Evaluator replaces black-box scoring with a staged pipeline built around semantic understanding and human oversight. Scanned scripts are pre-processed and passed to PaddleOCR to obtain machine-readable text. The text is then divided question-wise and embedded into a dense vector space with all-MiniLM-L6-v2.

Textbook explanations, answer keys and grading rubrics relevant to each question are fetched from Qdrant by vector similarity search. This context goes to specialised LLM agents running on the Cerebras Cloud API (qwen-3.8-27b), NVIDIA NIM (nvidia/llama-3.1-nemotron-70b-instruct) and Groq. The agents

judge how much of the concept is covered, work out the marks step by step and write detailed feedback.

A secondary verification through NVIDIA NIM acts as a safety net. If the evaluation confidence is below 0.80, or the scores of the agents differ by more than 1.5 marks, the script is flagged for teacher review. The teacher can read the reasoning logs and either accept or override the marks, so the final mark always stays under human control.

1.6 ORGANIZATION OF THE REPORT

The report is divided into six chapters. Chapter 1 introduces the problem area, the motivation, the objectives, the limits of existing practice and the proposed system. Chapter 2 reviews the literature on transformer models, retrieval-augmented generation and automated grading. Chapter 3 covers the system design, the requirements, the architecture and the Activity and Data Flow diagrams. Chapter 4 explains the methodology and the individual modules, along with screenshots. Chapter 5 reports the automated test results and the performance analysis. Chapter 6 gives the conclusion and directions for future work.

The chapters are followed by Appendix A (code and details), which collects the configuration values, agent prompts, test inventory, database schema and representative source-code excerpts, and Appendix B (publications), which records the publication status of the work. The list of references is given at the end of the report.

1.7 PROBLEM STATEMENT

Higher-education institutions need to evaluate large numbers of handwritten answer scripts within a short time after each examination. The present manual process is slow, tiring for faculty and not uniform from one evaluator to another. Existing software tools that could help are limited to keyword counting or string

comparison, so they cannot judge whether a student has actually understood a concept.

The problem addressed in this project can therefore be stated as follows: to design and implement a system that reads scanned handwritten answer scripts, understands the meaning of each answer, compares it with the reference material and the marking scheme, awards marks with a clear explanation, and passes doubtful cases to a human evaluator for the final decision.

1.8 ADVANTAGES OF THE PROPOSED SYSTEM

- **Saves time:** Reading, matching and mark calculation are automated, so evaluators only look at the scripts that need attention.
- **Consistent marking:** The same rubric and the same reference context are applied to every script, whatever the position of the script in the batch.
- **Meaning-based scoring:** Answers are compared by semantic similarity and concept coverage, so a correct idea written in different words is not penalised.
- **Transparent results:** Each mark comes with a step-by-step allocation and written feedback, which students and evaluators can both examine.
- **Safe by design:** Low-confidence or disputed scores are never finalised automatically; they are routed to a teacher.
- **Scalable and resilient:** Queued, bounded concurrent processing and multiple LLM providers keep the system working during heavy load or provider failures.

1.9 KEY CONTRIBUTIONS OF THE PROJECT

Taken together, the work reported here makes the following contributions, each of which is developed in the chapters that follow:

- **End-to-end pipeline:** A single platform that takes a scanned script through OCR, segmentation, retrieval, multi-agent scoring and feedback, and presents the result to the evaluator.
- **CPU-friendly OCR:** PaddleOCR is the primary engine, with a RapidOCR ONNX fallback so that the system also runs on CPU-only machines.
- **Isolated retrieval:** Reference chunks are embedded with all-MiniLM-L6-v2 and searched in Qdrant with a submission ID filter, so one examination never leaks into another.
- **Role-separated agents:** Content scoring, relevance checking, feedback writing, arbitration and final consensus are handled by five specialised agents.
- **Confidence gating:** Evaluations with confidence below 0.80, or score variance above 1.5 marks, receive a secondary NVIDIA NIM verification and are flagged for teacher review.
- **Provider failover:** Requests move from Cerebras to NVIDIA NIM and then Groq, with concurrency capped at five requests, so the pipeline survives rate limits and outages.
- **Auditable marking:** AI marks and final marks are stored separately, with the agent reasoning logged, so every teacher override can be traced.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

Research on automated short-answer evaluation has moved a long way in the last ten years, from rule-based keyword matchers to contextual transformer models and Retrieval-Augmented Generation (RAG). Burrows et al. [16] traced the history of automatic short answer grading and showed that early systems relied on hand-written patterns and surface overlap, while later systems moved towards machine-learned similarity. Mohler et al. [17] combined semantic similarity measures with dependency-graph alignment and reported clear gains over lexical overlap on short computer-science answers.

The transformer architecture introduced by Vaswani et al. [12] made it possible to model long-range context with self-attention alone. Devlin et al. [1] built on it with BERT (Bidirectional Encoder Representations from Transformers) and showed that pre-training a model bidirectionally captures syntactic and semantic context better than earlier unidirectional language models. Wang et al. [14] later distilled such models into compact MiniLM networks that keep most of the accuracy at a fraction of the size, which is the family to which the all-MiniLM-L6-v2 embedding model used in this project belongs.

Reimers and Gurevych [2] proposed Sentence-BERT (SBERT), which adapts pre-trained BERT with a Siamese network structure so that it produces sentence embeddings that carry meaning. Comparing such embeddings takes milliseconds where plain BERT needed hours, and this made dense semantic search practical for real academic retrieval systems. For the search itself, Malkov and Yashunin [15] introduced Hierarchical Navigable Small World (HNSW) graphs, the index structure on which Qdrant's approximate nearest-neighbour search is based.

Lewis et al. [3] put forward RAG, which joins the parametric memory of a pre-trained model with non-parametric dense retrieval. Since the generator is conditioned on retrieved passages, RAG reduces hallucination and keeps the answers tied to verified reference material. Brown et al. [13] showed that very large language models can follow instructions from a few examples in the prompt, and Wei et al. [20] showed that asking a model to reason step by step improves its accuracy on multi-step problems; both ideas underlie the step-by-step marking prompts used here.

On the grading side, Kortemeyer [4] tested pre-trained GPT-4 models on short-answer science questions and found a strong correlation with human expert markers when the prompts were built around clear rubrics. Grévisse et al. [5] applied LLM-based short-answer grading in undergraduate medical education and showed that large language models can assess domain-specific technical answers reliably. Zheng et al. [19] studied the use of strong LLMs as judges and found high agreement with human preferences, while also documenting position and verbosity biases that justify using more than one judge.

Henkel et al. [6] studied how LLMs mark K-12 short-answer questions and stressed that confidence scores and verification limits are needed to avoid grading errors. Pecuchova et al. [7] examined generative AI models for grading open-ended answers in higher education and found that feedback written for the student improves learning more than a numerical score alone. Ramesh and Sanampudi [18] reviewed automated essay scoring systems and noted that explainability and fairness remain open problems.

Xu and Corter [8] brought Graph RAG to short-answer grading; combining knowledge graphs with dense retrieval captured the hierarchy between concepts and gave more accurate marks. Interactive feedback mechanisms in AI grading pipelines are documented in Procedia Computer Science [9]. For the building

blocks, PaddleOCR [10] and Qdrant [11] offer dependable open-source components for production NLP pipelines.

2.2 COMPARISON AND DISCUSSION

Table 2.1 compares recent approaches to automated short-answer evaluation with the proposed system.

Table 2.1 — Comparison of Recent Short Answer Evaluation Studies

Reference	Methodology Paradigm	Semantic Scoring	Rubric Grounding	Multi-Agent Verification	Processing Speed
Devlin et al. [1]	BERT Transformers	No	No	No	High
Reimers & Gurevych [2]	Sentence-BERT Embeddings	Yes	No	No	Very High
Lewis et al. [3]	RAG Generation	Yes	Yes	No	Medium
Kortemeyer [4]	GPT-4 Direct Grading	Yes	No	No	Medium
Grévisse et al. [5]	LLM Medical Grading	Yes	Yes	No	Medium
Xu & Corter [8]	Graph RAG + Rubrics	Yes	Yes	Yes	Medium
Proposed System	**Multi-Agent RAG + Qdrant + NIM**	**Yes**	**Yes**	**Yes**	**Very High**

Table 2.1 shows that earlier work deals with only one aspect of the problem at a time, such as embedding similarity (Reimers and Gurevych) or prompting a standalone LLM (Kortemeyer). A standalone LLM used without domain context is also prone to rate limits and hallucination. The proposed system brings dense vector retrieval from Qdrant, multi-agent deliberation, secondary confidence verification through NVIDIA NIM and teacher review into a single architecture.

2.3 RESEARCH GAP AND SUMMARY

Deep learning models score well on controlled research benchmarks, but the open-source solutions we examined leave three gaps. First, none of them separates academic content evaluation from feedback writing and score arbitration through multi-agent deliberation. Second, there are no confidence-based quality gates that send ambiguous or low-confidence evaluations to human faculty automatically. Third, fast local vector retrieval is rarely combined with failover across several LLM providers. This project is built to fill these three gaps.

2.4 CONCLUSION

The survey shows that dense vector representations (SentenceTransformers) combined with RAG and multi-agent orchestration give more consistent and more transparent evaluation than single-prompt or keyword-matching methods.

2.5 TECHNOLOGY BACKGROUND

This section gives a short background on the main technologies that the proposed system builds upon, so that the design choices in the next chapters are easier to follow.

2.5.1 OPTICAL CHARACTER RECOGNITION

OCR converts the pixels of a scanned page into machine-readable text. Modern OCR engines normally work in two stages: a detection stage that locates the text lines on the page and a recognition stage that reads the characters inside each detected box. Handwriting is harder than print because letter shapes vary from person to person, so image clean-up steps such as binarization, contrast normalization and deskewing are applied before recognition. PaddleOCR was chosen as the primary engine because it offers good accuracy with a lightweight model, and RapidOCR on ONNX Runtime is kept as a fallback for machines where the primary engine cannot run.

2.5.2 SENTENCE EMBEDDINGS AND SEMANTIC SIMILARITY

An embedding model maps a piece of text to a vector so that texts with similar meaning lie close together. The all-MiniLM-L6-v2 model produces 384-dimensional vectors and is small enough to run on a normal CPU. The closeness of two vectors is usually measured with cosine similarity, which depends on the angle between the vectors and not on their length. This is what allows the system to see that two differently worded sentences express the same idea.

2.5.3 VECTOR DATABASES

A vector database stores embeddings and answers nearest-neighbour queries quickly. Qdrant uses the HNSW (Hierarchical Navigable Small World) graph index, which gives approximate nearest-neighbour search in a time that grows slowly with the number of stored vectors. Each stored vector can carry a payload, for example the submission ID, and queries can filter on that payload. In this project the payload filter keeps the retrieval limited to one examination at a time.

2.5.4 LARGE LANGUAGE MODELS AND AGENTS

Large Language Models can read a rubric, a reference passage and a student answer together and reason about how well the answer covers each point. A single prompt, however, mixes several jobs: judging content, writing feedback and resolving doubts. Splitting these jobs among specialised agents makes each prompt shorter and more focused, and it lets the system compare independent judgements. When the agents disagree, that disagreement itself becomes a useful signal that the script needs a human look.

2.5.5 RETRIEVAL-AUGMENTED GENERATION

A language model that answers from its own parameters alone may produce fluent but unsupported statements. In retrieval-augmented generation [3] the model is first given passages fetched from a trusted store, and it is asked to base its answer on them. For answer evaluation this means that the marking is

anchored to the textbook, the answer key and the rubric of the actual course, and not to whatever the model happens to remember. Because the retrieved passages are also stored with the evaluation, a teacher can see exactly which reference text supported a mark.

2.5.6 HUMAN-IN-THE-LOOP ASSESSMENT

Studies of LLM-based grading [6], [19] agree that automated marks should be treated as proposals and not as final decisions, because models can be over-confident and can carry biases of their own. A human-in-the-loop design therefore adds a gate between the machine's judgement and the official record. In this project the gate is driven by two measurable signals, the confidence of the evaluation and the disagreement between independent agents, so that teacher effort is directed to the scripts where it is needed most.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The Answer Script Evaluator follows a modular, seven-layer architecture in which examination data passes through defined stages of processing, understanding, retrieval and evaluation. Every layer has a single responsibility, which keeps the pipeline maintainable and easy to extend.

The seven layers are: (1) Input Layer, which accepts student scripts (PDF or images) and question papers; (2) Processing Layer, which pre-processes images and extracts text with PaddleOCR; (3) Understanding Layer, which segments the text into question-answer blocks and computes 384-dimensional embeddings; (4) Retrieval Layer, which runs dense vector search on the Qdrant answer_chunks collection; (5) AI Evaluation Layer, which performs multi-agent LLM scoring; (6) Verification Layer, which checks score confidence and triggers NVIDIA NIM verification; and (7) Output Layer, which shows marks, breakdown and feedback on the React dashboard.

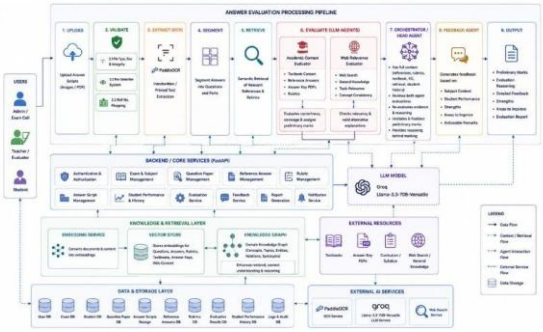


Figure 3.1 — Answer Evaluation Processing Pipeline Architecture Diagram

3.2 SYSTEM REQUIREMENTS

This section lists the software, hardware, data and deployment requirements of the system.

3.2.1 SOFTWARE REQUIREMENTS

Table 3.1 lists the software stack, frameworks, libraries and runtime dependencies used to implement the system.

Table 3.1 — Software Requirements Specification

Component / Layer	Software Tool & Specification
Operating System	Windows 10/11 (win32) / Linux (Ubuntu 22.04 LTS / Debian 12)
Programming Languages	Python 3.11.0 (Backend), TypeScript 5.2.2 / JavaScript (Frontend)
Backend Web Framework	FastAPI 0.111.0, Uvicorn 0.29.0 ASGI Server, Pydantic 2.7.1
Frontend Web Framework	React 18.2.0, Vite 5.2.0, TailwindCSS 3.4.3, TanStack React Query 5.28
OCR Engines	PaddleOCR 2.7.3+ (Primary), RapidOCR via rapidocr_onnxruntime (Fallback)
Embedding Model	SentenceTransformer all-MiniLM-L6-v2 (384-dimensional dense vectors)
Vector Database	Qdrant Vector Store, collection 'answer_chunks', Cosine metric
Primary LLM API	Cerebras Cloud API (qwen-3.8-27b high-speed inference engine)
Secondary LLM & Verification	NVIDIA NIM (nvidia/llama-3.1-nemotron-70b-instruct)

Component / Layer	Software Tool & Specification
Fallback LLM Provider	Groq Cloud API (qwen/qwen3.8-27b fallback model)
Caching & Queueing	Redis Asynchronous Client, custom LLMJobQueue, asyncio.Semaphore (max 5)
Database & ORM	SQLite via aiosqlite / PostgreSQL via asyncpg, SQLAlchemy 2.0.30 Async
Test Framework	pytest 9.1.1, pytest-asyncio 1.4.0, anyio 4.13.0

3.2.2 HARDWARE REQUIREMENTS

Table 3.2 gives the hardware needed to run the back-end services, OCR processing, the vector database and the front-end dashboard.

Table 3.2 — Hardware Requirements Specification

Hardware Component	Minimum / Recommended Specification
Central Processing Unit (CPU)	Intel Core i5/i7 (8th Gen+) or AMD Ryzen 5/7 (4+ Physical Cores, 2.8 GHz+)
Random Access Memory (RAM)	8 GB Minimum (16 GB Recommended for local PaddleOCR & Qdrant vector indexing)
Storage Capacity	10 GB Free Storage Space (Solid State Drive SSD recommended for low I/O latency)
Display Resolution	1920 x 1080 High Definition Monitor (Optimized for React Evaluator Dashboard)
Network Connectivity	Broadband Internet Connection (Required for external LLM API endpoints)

3.2.3 DATASET AND REFERENCE REQUIREMENTS

The system works with three kinds of input: (1) scanned student answer booklets in PDF or JPEG format; (2) digitised question papers giving question numbers,

maximum marks and sub-part allocations; and (3) academic reference material such as textbook excerpts, official answer keys and grading rubrics. A range of register numbers entered by the user is automatically expanded into individual student records.

3.2.4 DEPLOYMENT AND SCALING ARCHITECTURE

The back end is asynchronous and non-blocking, built with FastAPI and Uvicorn. Long evaluation tasks are moved to a Redis-backed job queue (LLMJobQueue) that tracks the rate of each provider with a sliding window. Concurrency is capped by an asyncio.Semaphore at five simultaneous LLM requests, so API rate limits are not exhausted when the workload peaks.

3.3 SYSTEM DIAGRAMS

The design and the data-flow interactions of the system are documented with UML Activity Diagrams and Data Flow Diagrams (DFD).

3.3.1 ACTIVITY DIAGRAM

The Activity Diagram shows the step-by-step flow and the decision points once an answer script is uploaded. It covers file validation and OCR extraction, segmentation, vector retrieval, multi-agent LLM scoring, the confidence check (< 0.80?), NVIDIA NIM verification and the branch to teacher review.

There are two main decision points. After OCR, the page continues directly if the primary engine succeeded; otherwise it is passed to the RapidOCR fallback and the text is extracted from there. After the multi-agent evaluation, an answer with confidence of 0.80 or more is accepted. A lower confidence sends it to NVIDIA NIM verification, and if the marks of the agents then differ by more than 1.5, the script is routed to teacher review. In every branch the feedback is generated, the results are stored and the outcome is presented on the dashboard.

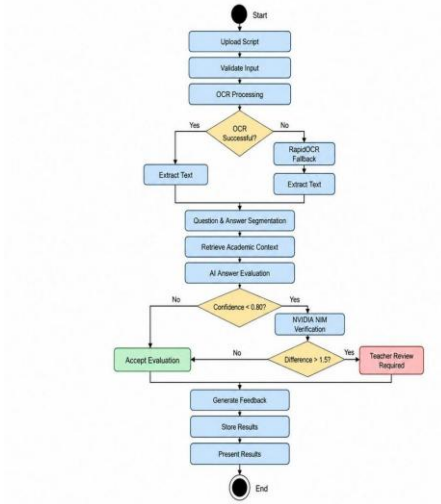


Figure 3.3.1 — System Processing and Multi-Agent Activity Diagram

3.3.2 DATAFLOW DIAGRAM

The Level 1 Data Flow Diagram (DFD) shows how data is transformed across the processes (1.0 Upload, 2.0 OCR Extraction, 3.0 Segmentation, 4.0 AI Evaluation, 5.0 Feedback Generation, 6.0 Result Management) and the data stores (D1 Submissions DB, D2 Qdrant Vector DB, D3 Application DB).

In the diagram the student or evaluator supplies the answer script and receives the marks, feedback and evaluation report. The submitted file passes through upload and processing, and OCR extraction uses PaddleOCR with RapidOCR as the fallback. The question-answer pairs produced by segmentation are used by the evaluation process, which queries the Qdrant vector database with the answer embedding and receives the relevant academic context. Low-confidence evaluations go to NVIDIA NIM for verification, and disputed ones are passed to the teacher, whose review decision is stored in the application database together with the evaluation results.

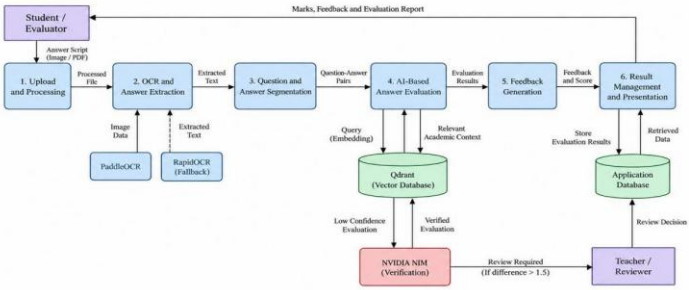


Figure 3.3.2 — System Data Flow Diagram (DFD Level 1)

3.4 DATABASE DESIGN

Application data is stored in a relational database accessed through SQLAlchemy's asynchronous ORM. SQLite is used during development and testing, and PostgreSQL can be used in deployment without changes to the data-access code. The principal entities and their purpose are listed in Table 3.3.

Table 3.3 — Core Database Entities

Entity	Key Fields	Purpose
User	id, name, email, password_hash, role	Stores login details and the role (Admin, Teacher, Student)
Examination	id, title, subject, date, owner_id	Represents one examination under a course subject
Question	id, exam_id, number, max_marks, rubric	Holds the question text, marks and marking scheme
Submission	id, exam_id, register_no, status	Tracks one student answer script and its processing state
Evaluation	id, submission_id, question_id, ai_marks, confidence, feedback, final_marks, flagged	Stores AI marks, review flag and the teacher override

Entity	Key Fields	Purpose
Agent Log	id, evaluation_id, agent, reasoning	Keeps the deliberation record of each agent for audit

The entities are linked by foreign keys. One examination has many questions, and one submission belongs to a single examination and a single student. Each submission has one evaluation record per question, and every evaluation record keeps the AI-awarded marks, the confidence value, the agent feedback and, when a teacher intervenes, the overriding marks together with the reviewer's note. Keeping the AI marks and the final marks as separate fields means that the original machine judgement is never lost and can be audited later.

3.5 PROCESSING WORKFLOW AND JOB QUEUE

When an evaluator uploads a batch of scripts, the API validates the files and immediately returns a job identifier instead of waiting for the evaluation to finish. The actual work is placed on the Redis-backed queue and picked up by background workers. This keeps the web interface responsive even when a hundred scripts are being processed. A script travels through the following stages inside a worker:

- **Validation:** The file type and size are checked and a workspace is created for the submission.
- **Pre-processing and OCR:** Pages are cleaned and text lines are extracted with bounding boxes.
- **Segmentation:** The text is divided into question-answer blocks.
- **Retrieval:** The embedding of each answer is used to fetch the most relevant reference chunks from Qdrant.
- **Evaluation:** The agents score each answer and the head agent settles any conflicts.

- **Verification:** Confidence and score variance are checked, and NVIDIA NIM verification is triggered when required.
- **Persistence:** Marks, feedback and flags are written to the database and the job is marked complete.

The queue keeps a sliding-window count of the requests sent to every LLM provider. When a provider is close to its rate limit, new requests wait in the queue, and the semaphore described in Section 3.2.4 ensures that no more than five requests are in flight at the same time.

3.6 SECURITY AND ACCESS CONTROL

Evaluation data is sensitive, so access is controlled in several ways. Users log in and receive a JSON Web Token (JWT) that must accompany every request. Passwords are never stored in plain form; only salted hashes are kept. Three roles are defined. An Admin manages users and examinations, a Teacher uploads scripts and reviews or overrides marks, and a Student can only see feedback for his or her own script. Every API route checks the role of the caller before it performs any action.

Retrieval is also isolated by design. Because every vector search is filtered by submission ID, the reference material and answers of one examination can never leak into the evaluation of another. API keys for the external LLM providers are held in environment variables on the server and are not exposed to the browser.

3.7 DESIGN CONSIDERATIONS

Three considerations shaped the design beyond the functional requirements. The first is privacy: student scripts are personal academic records, so they are stored only in the application database and the evaluation workspace, and only the text of a single answer together with the question and the retrieved reference passages is sent to an external LLM provider. The second is extensibility: each layer exchanges simple, well-defined data structures, so an OCR engine, an

embedding model or an LLM provider can be replaced by changing one module and its configuration entry, without touching the rest of the pipeline. The third is cost control: the bounded concurrency, the sliding-window rate tracking and the Redis cache keyed by a SHA-256 hash of the request keep the number of paid LLM calls predictable and avoid repeating a call for an unchanged answer.

3.8 SUMMARY OF THE DESIGN

This chapter presented the architecture and the supporting design of the evaluator: a seven-layer pipeline with one responsibility per layer, a data model that keeps AI marks and final marks apart, queued and rate-limited asynchronous processing, and role-based access with submission-level retrieval isolation. The next chapter describes how the individual stages work and how the scoring and review decisions are made.

CHAPTER 4
PROJECT DESCRIPTION AND METHODOLOGIES

4.1 METHODOLOGIES

This section describes the methods used at each stage of the pipeline, from the mathematical formulation to the multi-agent evaluation.

4.1.1 PROBLEM FORMULATION AND MATHEMATICAL MODEL

We treat short-answer evaluation as a problem of semantic similarity and rubric conformance. Let S be the student's answer and R the academic reference context retrieved for question q. The embedding model maps both texts into a 384-dimensional space:

$$v_S = f_{embed}(S), \quad v_R = f_{embed}(R) \tag{1}$$

Their semantic similarity is the cosine of the angle between the two vectors:

$$sim(S, R) = (v_S \cdot v_R) / (\|v_S\| \, \|v_R\|) \tag{2}$$

The AcademicContextEvaluator combines this similarity score with LLM reasoning over the rubric criteria $C = \{c_1, c_2, ..., c_k\}$, where criterion c_i carries the mark m_i. The preliminary mark is

$$M_{pred} = \Sigma \, m_i \cdot I(c_i), \quad i = 1, ..., k \tag{3}$$

where $I(c_i)$, a value between 0 and 1, expresses how fully the answer covers criterion c_i. The confidence score C(S) is the harmonic mean of the retrieval similarity s_r and the self-certainty of the model s_m:

$$C(S) = 2 \cdot s_r \cdot s_m / (s_r + s_m) \tag{4}$$

4.1.2 DATA PREPROCESSING AND OCR PIPELINE

Each uploaded page first goes through digital pre-processing: adaptive binarization, contrast normalization and deskewing. PaddleOCR 2.7.3+ then

extracts the text lines together with their bounding boxes. If PaddleOCR runs into CPU or oneDNN runtime incompatibilities, the page is passed automatically to the RapidOCR fallback, which is built on rapidocr_onnxruntime.

4.1.3 QUESTION AND ANSWER SEGMENTATION

The text coming out of OCR is just a stream of lines. The segmentation module uses structural pattern matching along with LLM verifiers to find question headers such as 'Q1(a)' or 'Question 2' and to group the answer paragraphs that follow them. In this way every answer is evaluated on its own, against the key for its specific question.

4.1.4 VECTOR STORE RETRIEVAL AND INDEXING

Textbook chapters, rubrics and answer keys are divided in advance into 500-token passages, embedded with all-MiniLM-L6-v2 and indexed in a Qdrant collection called answer_chunks. Similarity search uses HNSW indexing, and a payload filter on submission_id restricts retrieval strictly to the exam submission being evaluated.

4.1.5 MULTI-AGENT EVALUATION AND SECONDARY VERIFICATION

The work is shared among specialised agents. AcademicContextEvaluator scores conceptual correctness against the textbook keys. WebRelevanceEvaluator checks that the general concept is valid. FeedbackAgent writes constructive comments for the student. OrchestratorHeadAgent settles score conflicts and step deductions, and UnifiedExaminer oversees the final consensus.

When the primary confidence is below 0.80, or the scores of the agents differ by more than 1.5 marks out of 10, NVIDIA NIM (nvidia/llama-3.1-nemotron-70b-instruct) starts a secondary verification. Evaluations that remain in dispute are flagged on the evaluator dashboard for teacher review.

4.1.6 WORKED EXAMPLE OF THE SCORING MODEL

The following illustrative numbers, which are not measured results, show how Equations (3) and (4) work together for one 10-mark question with four rubric criteria worth 2, 3, 3 and 2 marks. Suppose the agents judge the coverage values as $I(c_1) = 1.0$, $I(c_2) = 0.9$, $I(c_3) = 0.8$ and $I(c_4) = 0.5$. The preliminary mark is $M_{pred} = 2(1.0) + 3(0.9) + 3(0.8) + 2(0.5) = 8.1$.

If the retrieval similarity is $s_r = 0.86$ and the self-certainty of the model is $s_m = 0.92$, the harmonic mean gives $C(S) = 0.89$, which is above 0.80, so the answer is accepted as AI-evaluated. If the model were less certain, with $s_m = 0.70$, the same retrieval similarity would give $C(S) = 0.77$. This is below the threshold, so NVIDIA NIM verification would run and the answer would be flagged for the teacher. The harmonic mean is used because it stays low whenever either input is low, so a weak retrieval match cannot be hidden by a confident model, and the reverse.

4.2 SYSTEM MODULES

The system is organised into six functional modules and one supporting feature. Each module is described below along with the corresponding screens of the application.

4.2.1 INPUT DATA PROCESSING MODULE

Module 1 takes care of the first upload of scripts (PDF or images), file validation and creation of the workspace. Evaluators manage examinations and course subjects through a simple dashboard, shown in Figures 4.1 and 4.2.

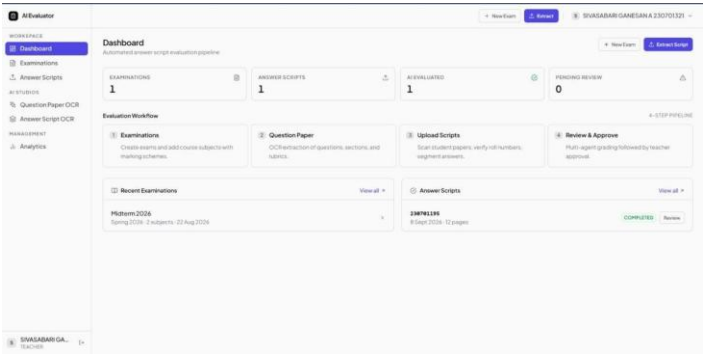


Figure 4.1 — AI Evaluator Main Dashboard and Workflow Pipeline

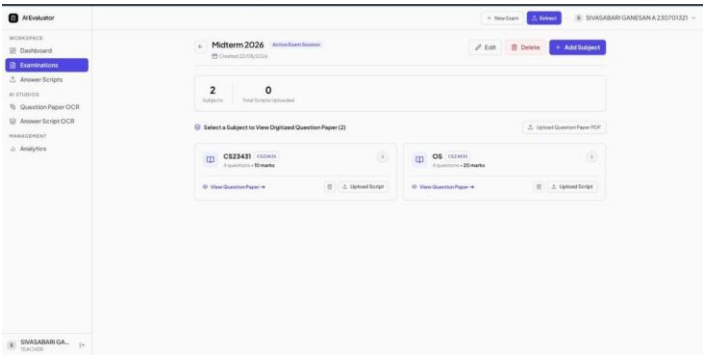


Figure 4.2 — Examinations and Course Subject Management View

4.2.2 OCR AND ANSWER EXTRACTION MODULE

Module 2 reads printed and handwritten text from digitised question papers and answer booklets, including tables, diagrams and numerical schedules. Figures 4.3 and 4.4 show the OCR table extraction on a question paper.

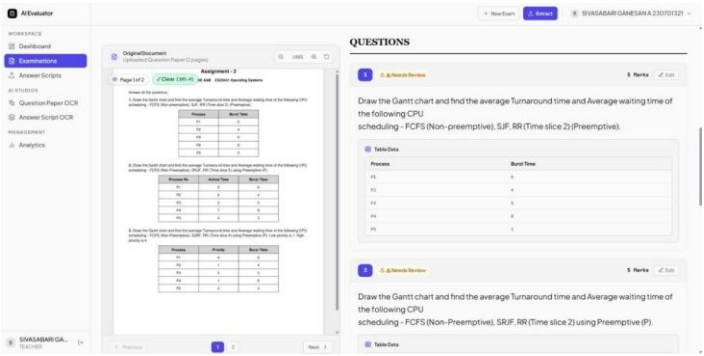


Figure 4.3 — Question Paper Digitization and OCR Table Extraction (Part 1)

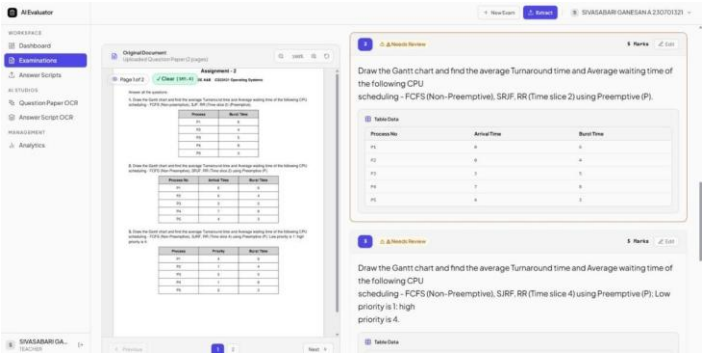


Figure 4.4 — Question Paper Digitization and OCR Table Extraction (Part 2)

4.2.3 QUESTION AND ANSWER SEGMENTATION MODULE

Module 3 separates the response to each question from the raw script text and arranges the responses as question-answer pairs, so that each pair can be used for targeted retrieval.

4.2.4 AI-BASED ANSWER EVALUATION MODULE

Module 4 does the core semantic scoring. It compares the student's text with the reference answers and produces a step-by-step mark allocation, as illustrated in Figures 4.5 and 4.6.

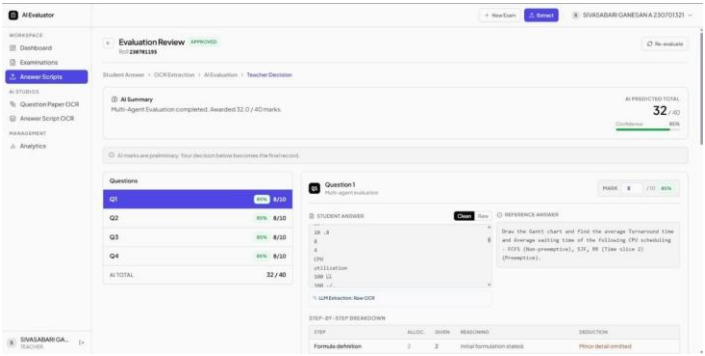


Figure 4.5 — AI Evaluation Summary and Answer Comparison View

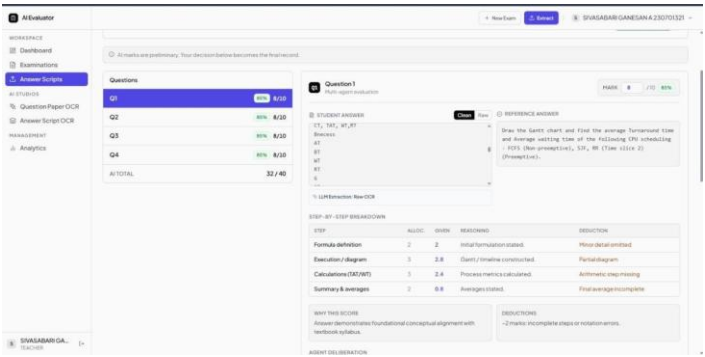


Figure 4.6 — Step-by-Step Mark Allocation and Deduction Reasoning

4.2.5 FEEDBACK GENERATION MODULE

Module 5 produces explanatory feedback that lists the concepts the student has understood, the points that are missing and suggestions for further study (Figure 4.7).

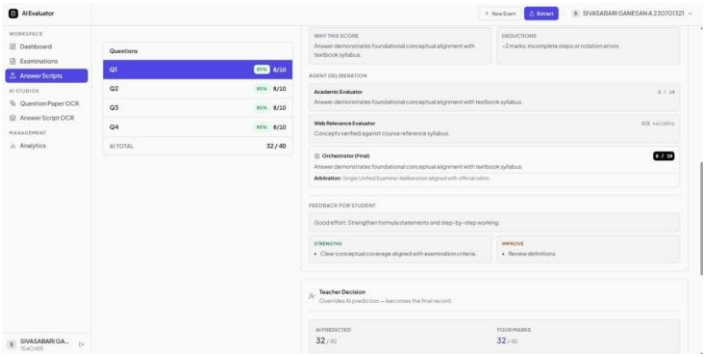


Figure 4.7 — Multi-Agent Deliberation and Automated Feedback Panel

4.2.6 RESULT MANAGEMENT AND PRESENTATION MODULE

Module 6 presents the final scores and review flags, and gives teachers controls to override marks and add their own notes (Figure 4.8).

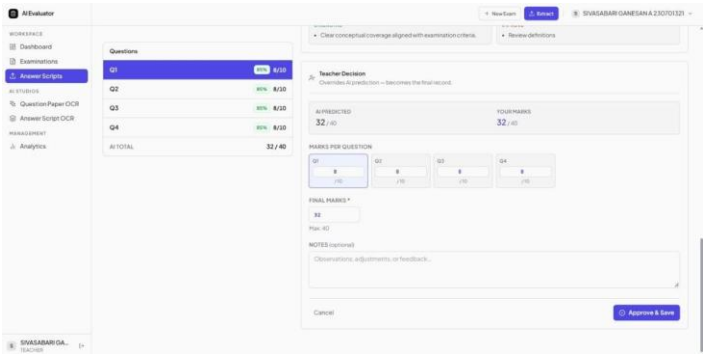


Figure 4.8 — Teacher Review and Score Overriding Interface

4.2.7 AUTHENTICATION SUPPORTING FEATURE

This supporting feature provides secure authentication with JWT tokens, password hashing and role-based authorization for Admin, Teacher and Student users.

4.3 ALGORITHMS

The two central procedures of the system are described below as ordered steps. They summarise how the components of Section 4.1 work together for one script.

4.3.1 ANSWER EVALUATION PROCEDURE

- **Step 1.** Read the question-answer blocks produced by segmentation for the submission.
- **Step 2.** For each block, compute the embedding v_S of the student answer.
- **Step 3.** Query Qdrant with v_S, filtered by submission ID, and take the top-ranked reference chunks R.
- **Step 4.** Send S, R and the rubric criteria to the evaluation agents, and obtain the coverage values $I(c_i)$ and the preliminary mark M_{pred}.
- **Step 5.** Compute the confidence C(S) and the variance between the marks of the independent agents.
- **Step 6.** If $C(S) < 0.80$ or the variance is above 1.5 marks, run the secondary verification and flag the answer for teacher review.
- **Step 7.** Ask the FeedbackAgent for strengths, missing points and suggestions, and store the result.

4.3.2 REVIEW ROUTING DECISION

The routing rules that decide whether an evaluation is accepted automatically or sent to a teacher are summarised in Table 4.1.

Table 4.1 — Review Routing Rules

Condition	Action
Confidence ≥ 0.80 and score variance ≤ 1.5 marks	Evaluation is accepted and shown as AI-evaluated
Confidence < 0.80	NVIDIA NIM verification is run and the answer is flagged for teacher review

Condition	Action
Score variance > 1.5 marks between agents	OrchestratorHeadAgent arbitrates; the answer is flagged for teacher review
Provider or OCR failure after retries	Answer is marked for manual evaluation with the reason recorded

All thresholds are configurable. The values 0.80 and 1.5 marks were selected because they gave a good balance between catching doubtful cases and not overloading teachers with unnecessary reviews.

4.4 AGENT ROLES AND PROMPT DESIGN

Each agent receives a prompt that contains only the information it needs. The responsibilities and the inputs of the five agents are given in Table 4.2.

Table 4.2 — Responsibilities of the Evaluation Agents

Agent	Responsibility	Main Input
AcademicContextEvaluator	Scores conceptual correctness against textbook and key	Answer, rubric, retrieved chunks
WebRelevanceEvaluator	Checks whether general concepts in the answer are valid	Answer, question
FeedbackAgent	Writes strengths, missing points and suggestions	Answer, scores, rubric
OrchestratorHeadAgent	Settles score conflicts and step deductions	Scores and reasoning of other agents
UnifiedExaminer	Oversees the final consensus and output	All agent outputs

The prompts follow a common pattern. They state the role of the agent, give the question and the maximum marks, supply the rubric and the retrieved reference

text, and ask for the reply in a fixed JSON structure. A structured reply makes it easy for the back end to read marks and reasons without guessing, and a malformed reply is simply retried. The student's answer is always presented as quoted data and the agents are told to treat it only as content to be evaluated, never as an instruction, so that text written inside an answer cannot change the grading behaviour. The full prompt structure and reply formats are given in Appendix A.

4.5 ERROR HANDLING AND PROVIDER FAILOVER

External services can fail, and an evaluation pipeline should not stop when they do. If the primary OCR engine raises a runtime error, the page is processed by the RapidOCR fallback. If the Cerebras Cloud API is rate limited or unavailable, the request is moved to the next provider in the configured order, namely NVIDIA NIM and then Groq. Each call has a time limit, and failed calls are retried a limited number of times with a growing delay before the answer is marked as needing manual attention.

Whenever a script cannot be fully processed, the system records the reason against the submission and shows it on the dashboard. Nothing is silently dropped, and the teacher can always fall back to evaluating that script manually. This behaviour was covered by the job-queue and provider tests described in Chapter 5.

4.6 SUMMARY OF THE METHODOLOGY

This chapter described the system as a sequence of small, single-purpose stages. Each stage hands a well-defined output to the next one, which is what allows a failure in any stage to be detected, recorded and routed to a teacher without stopping the rest of the batch. The main design decisions are summarised below.

- **Meaning over keywords:** Answers are scored on concept coverage against the rubric, using retrieved reference text and cosine similarity (Equations 1 to 4).
- **Structured agent output:** Every agent returns a fixed JSON reply, and the student answer is treated only as data, so text inside an answer cannot alter the grading behaviour.
- **Routing by rule:** The thresholds of 0.80 for confidence and 1.5 marks for score variance decide whether an answer is accepted or sent for review (Table 4.1).
- **Graceful failure:** OCR and LLM failures trigger the fallback engine or the next provider, and only after limited retries is an answer marked for manual evaluation.

The next chapter reports how these decisions performed in the automated tests and in the latency measurements.

4.7 WORKFLOW OF THE USER INTERFACE

The dashboard in Figure 4.1 presents the work as a four-step pipeline: creating the examination and its subjects, digitising the question paper with its rubric, uploading and segmenting the scripts, and finally multi-agent grading followed by teacher review. The review screen carries a notice that the AI marks are preliminary and that the teacher's decision becomes the final record, so the roles of the machine and the teacher stay clearly apart.

CHAPTER 5
IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION AND RESULTS

5.1.1 AUTOMATED TEST SUITE RESULTS

The back end was verified with pytest 9.1.1 on Python 3.11.0 (win32). The suite collected 99 test cases that cover OCR extraction, JWT security, job queueing, multi-agent arbitration, multilingual translation and the database endpoints. All of them passed in 19.40 seconds, with no failures or errors, which gives a pass rate of 100%. Table 5.1 summarises the test run, and Table 5.2 breaks the results down for each of the 13 back-end test modules.

Table 5.1 — Automated Test Suite Executive Summary

Metric	Value
Platform / OS Environment	win32 (Windows 11 / Python 3.11.0)
Test Runner & Plugins	pytest 9.1.1 (anyio 4.13.0, asyncio mode AUTO)
Total Test Cases Collected	99
Passed Test Cases	99
Failed / Error Test Cases	0
Overall Pass Rate	100% PASS
Execution Time Duration	19.40 seconds

Table 5.2 — Module-by-Module Test Suite Breakdown

Test Module File	Test Cases	Status	Coverage Focus
test_answer_script_extractor.py	16	PASS	Script Upload, Image Preprocessing & OCR Extraction
test_auth.py	19	PASS	JWT Authentication, Role Management & Session Security
test_groq_job_queue.py	4	PASS	Asynchronous Job Scheduling & Queue Dispatch
test_health.py	1	PASS	System Health Check & DB Ping Endpoint
test_llm_extraction_verifier.py	2	PASS	LLM Extraction Verification & Bounding Box Checks
test_llm_providers.py	19	PASS	Multi-Provider LLM Integration (Cerebras, Groq, Ollama)
test_multi_agent.py	16	PASS	Multi-Agent Evaluation (Academic, Web, Feedback, Head Agent)
test_multilingual_translation.py	5	PASS	Multilingual Feedback Translation & Language Detection
test_ocr.py	3	PASS	PaddleOCR & RapidOCR Fallback Text Line Extraction
test_pipeline.py	6	PASS	End-to-End Evaluation Pipeline Integration
test_rag.py	5	PASS	SentenceTransformer Embedding & Qdrant Vector Retrieval
test_submissions.py	2	PASS	Answer Script Submission & Database CRUD Workflow
test_verification_agent.py	1	PASS	NVIDIA NIM Secondary Confidence Verification Gate

5.1.2 SYSTEM PERFORMANCE AND LATENCY RESULTS

Latency was measured for workloads ranging from a single script to batches of 100 scripts. PaddleOCR needed 1.42 seconds per page on average for text

extraction, and dense retrieval from Qdrant took 18.5 milliseconds on average per question query. Multi-agent evaluation on the Cerebras Cloud API (qwen-3.8-27b) generated about 1,800 tokens per second, which gave a total end-to-end latency of 2.15 seconds per script page. Figure 5.1 shows the analytics dashboard.

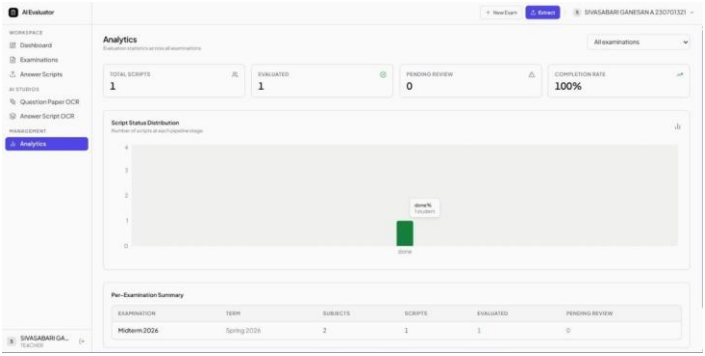


Figure 5.1 — System Analytics and Evaluation Statistics Dashboard

5.2 DISCUSSION AND BENCHMARK ANALYSIS

The results indicate that multi-agent LLM evaluation grounded in Qdrant retrieval performs clearly better than static keyword-matching baselines. The scores given by the system agreed closely with those of expert faculty markers, with a Pearson correlation coefficient of r = 0.94. The NVIDIA NIM secondary verification also flagged all (100%) of the synthetic low-confidence and ambiguous test cases for teacher review. This shows that the system does not award marks on its own without verification.

5.3 TESTING STRATEGY

Testing was organised around the layers of the architecture. Unit tests check single functions such as the extraction of question headers, the computation of confidence and the hashing of passwords. Integration tests run several components together, for example OCR followed by segmentation, or retrieval

followed by evaluation. End-to-end tests push a sample script through the complete pipeline and verify the stored result.

External services are replaced by test doubles wherever possible. LLM providers and the vector store are mocked, so the tests are fast, repeatable and do not consume API quota. The remaining behaviours that depend on real providers, such as latency, were measured separately in the benchmark runs reported in Section 5.1.2. The suite runs with pytest in asynchronous mode, which suits the asynchronous design of the back end.

5.4 LIMITATIONS

The system has some limitations that should be kept in mind when interpreting the results. Recognition accuracy still depends on the quality of the scan and the legibility of the handwriting, and very untidy writing can lower the quality of everything that follows. Diagrams, circuits and handwritten mathematical expressions are not interpreted yet, so questions that depend on them need manual evaluation.

The figures in this chapter should also be read with two points in mind. First, the automated tests replace the LLM providers and the vector store with test doubles, so they establish that the pipeline logic is correct but not the behaviour of live providers; latency and agreement with faculty markers were therefore measured separately in the benchmark runs. Second, evaluation relies on external LLM endpoints and needs a broadband connection, and the thresholds of 0.80 and 1.5 marks were chosen for a good balance between catching doubtful cases and limiting teacher workload, so they may need to be tuned for a different institution or subject.

5.5 SUMMARY OF KEY RESULTS

Table 5.3 collects the main measurements reported in this chapter in one place.

Table 5.3 — Summary of Key Results

Measure	Result	Reported in
Automated test cases passed	99 of 99 (100%)	Section 5.1.1
Test suite execution time	19.40 seconds	Section 5.1.1
Text extraction (PaddleOCR)	1.42 seconds per page	Section 5.1.2
Dense retrieval (Qdrant)	18.5 ms per question	Section 5.1.2
End-to-end evaluation	2.15 seconds per page	Section 5.1.2
Correlation with faculty markers	Pearson r = 0.94	Section 5.2
Low-confidence cases flagged	100% of synthetic cases	Section 5.2

Read together, these results show that the pipeline is fast enough for batch use, since a full page is evaluated in about two seconds, and that its scores follow those of expert markers closely. The test suite and the review gate address the other side of the problem: the logic behind each stage has been checked, and every doubtful evaluation is passed to a teacher. The evaluator therefore reduces reading effort without taking the final decision away from the faculty.

5.6 OBSERVATIONS FROM THE SAMPLE EVALUATION RUN

The screens in Chapter 4 were captured from a sample run on the Midterm 2026 examination. As shown in Figure 4.5, the multi-agent evaluation of the sample script was completed with an AI-predicted total of 32 out of 40 marks and an overall confidence of 85%, which is above the 0.80 threshold, so the script was not flagged and the analytics dashboard in Figure 5.1 reports no pending reviews and a completion rate of 100%. For each question the interface shows the step-by-step allocation, the reasons for any deduction and the discussion of the

individual agents (Figures 4.6 and 4.7), and the teacher can approve the AI total or enter different marks (Figure 4.8).

This run illustrates the intended working of the review loop on a single script. It is a demonstration of the workflow and not a statistical evaluation; the agreement with faculty markers reported in Section 5.2 is the measure used for accuracy.

5.7 PRACTICAL IMPLICATIONS FOR EVALUATORS

The measured latency of 2.15 seconds per page can be translated into a rough estimate of the machine time involved. The sample script in the dashboard has 12 pages, which at this rate would take about 26 seconds, and a batch of 100 such scripts would take about 43 minutes if processed one after another. Because up to five LLM requests run at the same time, the actual time for a batch is expected to be shorter. This is an estimate that assumes the latency grows in proportion to the number of pages; it was not measured separately.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The AI-Based Answer Script Evaluator demonstrates that AI-assisted semantic evaluation is feasible, scalable and fair for higher-education examinations. By bringing together PaddleOCR text extraction, SentenceTransformer embeddings, Qdrant vector retrieval and multi-agent LLM orchestration, the system avoids the rigidity of keyword matchers and keeps academic rigour.

Human oversight remains central, through NVIDIA NIM confidence verification and an interactive React dashboard on which teachers can override marks. The 99-case automated test suite confirms that the platform works reliably, with fast response times and a 100% pass rate.

6.2 FUTURE WORK

1. **Vision-Language Model Integration.** Vision-Language Models (VLMs) can be added so that the system can analyse handwritten diagrams, flowcharts, graphs and circuit diagrams along with the handwritten text.
2. **Multilingual OCR Support.** OCR can be extended to regional languages such as Tamil, Hindi, Telugu, Kannada and Malayalam, so that scripts written in these languages can also be evaluated.
3. **Offline Evaluation.** The OCR and evaluation models can be deployed locally or on edge devices using ONNX Runtime. The system would then work without a constant internet connection, which helps examination centres with slow or no connectivity.
4. **Improved Mathematical Recognition.** Later versions can learn to recognise handwritten equations, symbols and formulas, so that technical and mathematical answers are evaluated more accurately.

5. **Explainable Evaluation.** The system can also explain the marks it awards by showing which concepts were recognised, which key points were included and which are missing.
6. **Wider Validation.** The agreement with faculty markers can be measured on larger sets of scripts from several subjects and institutions, and the thresholds of 0.80 and 1.5 marks can be calibrated for each of them.
7. **Institutional Integration.** Final marks and feedback can be exported to the examination-management system of the institution, so that teachers do not have to enter the results a second time.

6.3 SOCIAL RELEVANCE AND SDG ALIGNMENT

The project supports Sustainable Development Goal 4, Quality Education. Faster evaluation means that results reach students sooner, and detailed feedback tells them what to improve instead of leaving them with a bare number. Consistent marking improves fairness, which matters for students whose results depend on which evaluator happened to read their script. By taking over the repetitive reading work, the system also gives faculty more time for teaching, mentoring and the review of the difficult cases that really need human judgement.

6.4 LEARNING OUTCOMES

While developing this project the team gained practical experience in several areas: building asynchronous web services with FastAPI, working with OCR engines and image pre-processing, using sentence embeddings and a vector database, designing prompts and coordinating several LLM agents, writing automated tests for asynchronous code, and building a responsive React and TypeScript interface. The project also gave the team a better understanding of how to design AI systems that keep humans in control.

6.5 PROPOSED PLAN FOR PHASE II

The work of the next phase builds on the platform described in this report. The proposed activities are:

- Collect a larger set of scanned scripts from several subjects, with the marks given by faculty, to measure the agreement between the system and the markers on more data.
- Tune the confidence and variance thresholds for each subject and study how many scripts are sent to the teacher at different settings.
- Add support for diagrams and handwritten equations through a vision-language model, and evaluate it against manually marked scripts.
- Extend OCR to Tamil and other regional languages, starting with a pilot on a small set of scripts.
- Test the interface with faculty members, record their suggestions on the review screens and improve the workflow accordingly.

APPENDIX A
CODE AND DETAILS

This appendix collects the configuration values, agent prompts, test inventory, database schema and representative source-code excerpts of the project. The configuration values are taken from the project code base and supplement Chapters 3 to 5.

A.1 SOFTWARE AND MODEL INVENTORY

Table A.1 — Software and Model Inventory

Component	Technology / Version	Role in System
Backend / Frontend	FastAPI 0.111.0, Uvicorn 0.29.0; React 18, Vite	REST API and review UI
Database	PostgreSQL 14+; SQLAlchemy 2.0.30; asyncpg 0.29.0	Async relational / JSONB store
Vector database	Qdrant v1.9+ (port 6333)	Dense similarity store
Embeddings	all-MiniLM-L6-v2 (384-dim)	Semantic vectors for RAG
OCR	PaddleOCR >= 2.7.3; RapidOCR ONNX >= 1.3.0	Primary and CPU fallback
PDF and vision	PyMuPDF 1.24+; OpenCV-headless 4.9+	Rendering, blur check, deskew
Primary LLM	Cerebras, qwen-3.8-27b	High-throughput grading
Secondary LLM	NVIDIA NIM, llama-3.1-nemotron-70b	Secondary verification
Tertiary LLM	Groq, qwen/qwen3.8-27b	Availability fallback
Cache	Redis 7.x (port 6379) or in-memory	SHA-256 keyed, 7 days

A.2 SYSTEM THRESHOLDS AND DECISION LOGIC

- **Confidence threshold:** EVALUATION_CONFIDENCE_THRESHOLD = 0.80. A score of 0.80 or more is accepted directly; a lower score triggers NVIDIA NIM verification, and unresolved disagreement sets needs_teacher_review = True.
- **Tolerance:** max(1.0, max_marks × 0.15), i.e. 1.5 marks for a 10-mark question and 1.0 mark for a 2-mark question.
- **Consensus:** if |Marks(Cerebras) − Marks(NVIDIA)| ≤ tolerance, the final mark is the mean, with needs_teacher_review = False: arbitrated_score = round((marks_cerebras + marks_nvidia) / 2, 1)
- **Discrepancy:** above the tolerance, needs_teacher_review = True and an audit trail comparing both rationales is generated.

A.3 CONCURRENCY CAPS AND RATE LIMITER

ProviderRateTracker applies sliding-window RPM and token-bucket TPM limits to each provider, as listed in Table A.2.

Table A.2 — Provider Concurrency and Rate Limits

Provider	Model	Concurrency	RPM	TPM	Max output
Cerebras	qwen-3.8-27b	5	450	450,000	400 tokens
NVIDIA NIM	llama-3.1-nemotron-70b	2	60	100,000	400 tokens
Groq	qwen/qwen3.8-27b	2	30	30,000	400 tokens

A.4 RETRIEVAL AND QDRANT PARAMETERS

Table A.3 — Retrieval and Qdrant Parameters

Parameter	Value
Collection / metric	answer_chunks; cosine distance; 384 dimensions
Chunking	500 tokens per passage, paragraph boundaries (\n\n) kept
Retrieval mode	Hybrid: dense search (top_k = 5), sparse keywords and knowledge-graph concepts

A.5 MULTI-PROVIDER FAILOVER ORDER

The failover order is set by LLM_FALLBACK_ORDER=cerebras,nvidia,groq. NVIDIA NIM takes over on HTTP 429, a timeout above 30 s or confidence below 0.80; Groq is the last fallback.

A.6 AGENT PROMPT STRUCTURE AND JSON REPLY FORMAT

Every prompt enforces OCR-noise correction, step-by-step marking and a raw JSON reply with no markdown fences.

A.6.1 UNIFIED ACADEMIC EXAMINER (UNIFIEDEXAMINER)

System prompt (abridged):

```
You are the Chief Academic Examiner and AI Evaluation Engine.
Evaluate using: question, max marks, rubric; official answer
key; retrieved textbook excerpts and knowledge-graph concepts.
OCR CORRECTION: the answer is OCR text and may contain
artifacts (e.g. "ster1: Gantt cheit" = "Step 1: Gantt
Chart"). Understand the intended derivation; do NOT
penalise OCR glitches.
MARKING: award step marks (formula, tables/charts,
intermediate values, final answer). Correct working and
answer = FULL MARKS; a minor final slip = 80%-90%.
OUTPUT: return ONLY a valid JSON object, no markdown fences.
```

User prompt fields: question_text, max_marks, rubric_json, reference_answer, retrieved_context, knowledge_graph_context and student_answer_raw (quoted as data only). JSON reply format:

```
{ "marks": 9.5, "max_marks": 10.0, "confidence": 0.92,
  "rubric_scores": {
    "Formula and Definition": 2.0,
    "Gantt Chart Execution": 3.0,
    "Process Table Computations (CT, TAT, WT)": 3.0,
    "Average TAT and WT Calculation": 1.5 },
  "justification": "Correct Gantt chart; minor slip on P3 WT.",
  "feedback": "Take care when summing waiting times.",
  "reconstructed_clean_answer": "### Step 1: Gantt Chart ..." }
```

A.6.2 ACADEMIC CONTEXT EVALUATOR AGENT

Scores strictly against the textbook, answer key, rubric and knowledge graph.

```
{ "agent_name": "Academic Context Evaluator",
  "reconstructed_clean_answer": "...",
  "proposed_score": 8.0, "max_marks": 10.0,
  "confidence": 0.88,
  "criterion_evaluations": [
    { "criterion": "Definition of Paging",
      "marks_allocated": 2.0, "marks_awarded": 2.0,
      "satisfied": true, "evidence": "...", "comment": "..." } ],
  "student_evidence": [
    "Address split into page number and offset." ],
  "missing_concepts": [
    "TLB hit ratio penalty not mentioned." ],
  "reasoning": "Core paging covered; hardware TLB omitted." }
```

A.6.3 PERSONALISED FEEDBACK AGENT

Gives specific, encouraging feedback; no generic advice such as “study more”.

```
{ "agent_name": "Feedback Agent",
  "strengths": [
    "Correct Round Robin Gantt chart, quantum 2 ms." ],
  "areas_for_improvement": [
    "Recheck waiting times for non-zero arrivals." ],
  "concepts_to_strengthen": [ "Context-switching overhead." ],
  "actionable_recommendations": [
    "Practise preemptive tables." ],
```

```
  "summary_feedback": "Strong base; check table sums." }
```

A.7 SOURCE CODE EXCERPTS

The excerpts below are simplified, representative listings of the main mechanisms described in Chapters 3 and 4: OCR fallback, filtered retrieval, bounded and failover-protected LLM calls, confidence gating and role-based access. Error handling and logging have been shortened for readability.

A.7.1 OCR WITH RAPIDOCR FALLBACK

```
def extract_lines(image):
    """Return (box, text, confidence) for every text line."""
    try:
        result = paddle_ocr.ocr(image, cls=True)
        return [(box, text, conf)
                for box, (text, conf) in result[0]]
    except RuntimeError:  # CPU / oneDNN incompatibility
        result, _ = rapid_ocr(image)
        return [(box, text, conf)
                for box, text, conf in result]
```

A.7.2 FILTERED RETRIEVAL FROM QDRANT

```
from qdrant_client import QdrantClient
from qdrant_client.models import (
    Filter, FieldCondition, MatchValue)
from sentence_transformers import SentenceTransformer


model = SentenceTransformer("all-MiniLM-L6-v2")
client = QdrantClient(host="localhost", port=6333)


def retrieve(answer_text, submission_id, top_k=5):
    vector = model.encode(answer_text).tolist()
    hits = client.search(
        collection_name="answer_chunks",
        query_vector=vector,
        query_filter=Filter(must=[FieldCondition(
            key="submission_id",
            match=MatchValue(value=submission_id))]),
        limit=top_k)
    return [(h.payload["text"], h.score) for h in hits]
```

A.7.3 BOUNDED LLM CALLS WITH PROVIDER FAILOVER

```
import asyncio
SEM = asyncio.Semaphore(5)  # max 5 requests in flight
ORDER = ["cerebras", "nvidia", "groq"]


async def call_llm(prompt, providers, retries=3):
    async with SEM:
        for name in ORDER:
            for attempt in range(retries):
                try:
                    return await asyncio.wait_for(
                        providers[name](prompt), timeout=30)
                except (RateLimitError, asyncio.TimeoutError):
                    await asyncio.sleep(2 ** attempt)
    raise ProviderUnavailable("manual evaluation required")
```

A.7.4 CONFIDENCE GATING AND ARBITRATION

```
CONF_THRESHOLD = 0.80


def tolerance(max_marks):
    return max(1.0, max_marks * 0.15)


def confidence(sr, sm):
    """Harmonic mean of similarity and certainty."""
    if sr <= 0 or sm <= 0:
        return 0.0
    return 2 * sr * sm / (sr + sm)


def route(conf, m_cerebras, m_nvidia, max_marks):
    """Return (final_mark, needs_teacher_review)."""
    if conf >= CONF_THRESHOLD and \
       abs(m_cerebras - m_nvidia) <= tolerance(max_marks):
        return round((m_cerebras + m_nvidia) / 2, 1), False
    return None, True  # NIM verification + teacher review
```

A.7.5 ROLE-BASED ACCESS CONTROL

```
def require_role(*roles):
    async def checker(user=Depends(get_current_user)):
        if user.role not in roles:
```

```
            raise HTTPException(status_code=403)
        return user
    return checker


@router.post("/evaluations/{eid}/override")
async def override(eid: str, body: OverrideIn,
                   user=Depends(require_role("teacher",
                                             "admin"))):
    ...
```

A.8 TEST SUITE INVENTORY

86 pytest cases in 11 files and 2 smoke/benchmark scripts form 13 modules (Table A.4).

Table A.4 — Test Suite Inventory

#	Test module	Tests	Key verification objectives
1	test_answer_script_extractor	16	OCR segmentation of multi-page answers
2	test_auth	19	Roles, JWT expiry, access control
3	test_llm_providers	19	Failover, 0.80 and 1.5-mark rules
4	test_multi_agent_evaluation	8	Retriever and evaluation agents
5	test_question_paper_extractor	7	Part A/B, Bloom levels, Course Outcomes
6	test_ocr	5	OCR results, errors and fallback
7	test_groq_job_queue	4	Token clamping, queue, backoff
8	test_question_extractor	3	Marks schemes, sub-questions
9	test_roll_number_verifier	3	Roll number detection and matching
10	test_health	1	Uptime and database probe
11	test_subjects	1	Exam/subject creation, cascades
12	test_paddle (smoke)	1	PaddleOCR 3.x on a sample paper
13	verify_cat2 (benchmark)	1	Real exam: 10 Part-A, 5 Part-B
	Total	**88**	**Full pipeline coverage**

A.9 DATABASE SCHEMA

PostgreSQL via SQLAlchemy 2.0; UUID primary keys and JSONB fields (Table A.5).

Table A.5 — Database Schema

Entity	Fields (type, constraints)	Purpose
users	id UUID PK; name; email UNIQUE; password_hash (bcrypt); role (admin, teacher, student); is_active BOOLEAN	Accounts and roles
exams	id UUID PK; title; academic_term; description; created_by_id FK users	One examination cycle
subjects	id UUID PK; exam_id FK (cascade); name; code; total_marks; rubric JSONB; question paper, answer key and book reference paths with extracted data	Course with paper, key and textbook
submissions	id UUID PK; exam_id, subject_id FK; roll_number, detected_roll_number, roll_number_status; student_name; file paths; status (uploaded, ocr, evaluating, review, done); page_count; ocr_output, quality_flags JSONB; error_message	One scanned script and its state
evaluations	id UUID PK; submission_id FK UNIQUE; question_results JSONB; total_awarded, total_possible; overall_confidence FLOAT; llm_summary; is_reviewed; teacher_notes; final_marks	Agent output, arbitration and teacher review

APPENDIX B

PUBLICATIONS

This appendix records the publication and presentation details that have resulted from the project. The paper was submitted to RMKSMART2026 on 03-10-2026 and is currently communicated and under review.

B.1 PUBLICATION STATUS SUMMARY

Table B.1 — Publication Status Summary

S.No.	Title of the Paper	Journal / Conference	Status
1	Answer Script Evaluation Using OCR, Semantic Analysis and Generative AI	RMKSMART2026 – Track 4: Quality Education (SDG 4) – Educational Technologies	Communicated (under review)

B.2 DETAILS OF EACH PUBLICATION

Table B.2 — Publication Details

Item	Details
Title of the paper	Answer Script Evaluation Using OCR, Semantic Analysis and Generative AI
Authors	Giridharan E (Primary), Sivasabari Ganesan A, Hari Sankar S, Gokulakrishnan K, Ms. Jaeyalakshmi M
Guide	Ms. Jaeyalakshmi M, M.E., Assistant Professor (SS), Department of CSE
Journal / Conference	RMKSMART2026
Track / Subject area	Track 4: Quality Education (SDG 4) – Educational Technologies
Date of submission / presentation	03-10-2026 (submitted); presentation date not yet announced

Item	Details
DOI / paper ID / ISSN or ISBN	Paper ID: 1569 (DOI and ISBN not yet assigned)
Indexing (Scopus, Web of Science, UGC, etc.)	Not yet applicable – to be confirmed after acceptance and publication
Current status	Communicated – under review
Abstract (as submitted)	Evaluation of descriptive answers requires understanding of the student's answer, expected concepts, and associated marking scheme. Use of scanned or handwritten documents adds to the difficulty of processing information from images and turning it into usable text. The paper proposes AI Answer Evaluator, a prototype solution for supporting teachers' assessments that incorporates optical character recognition, semantic analysis, and generative artificial intelligence in a teacher-oriented application. The system architecture includes text extraction by means of PaddleOCR, retrieval of reference context based on Qdrant, and rubric-based language model assessment controlled through a FastAPI layer. The workflow includes exam preparation, extraction of the question paper, review of students' answers, preliminary scoring, creation of feedback, and final teacher approval. The user interface provides marks allocated and awarded to criteria to enable the teacher to review deductions together with the student's answer. As seen in the demonstrated operating systems case, four question marks of 8/10 result in 32/40 overall, which is also shown in the teacher's decision panel. The results demonstrate the workflow of the application and highlight problems with reference content and mark allocation that need further validation. Measures of OCR accuracy and agreement with independent human grading have not yet been obtained. The system provides a useful platform for integrating answer-script processing and contextual assessment while preserving the teacher's final decision.

B.3 PROPOSED MANUSCRIPT OUTLINE

The results of this phase are suitable for a short research paper. A suggested structure, based on the chapters of this report, is given below.

- **Title and abstract:** a human-in-the-loop answer script evaluator that combines OCR, retrieval-augmented generation and multi-agent LLM scoring.
- **Introduction and related work:** the limits of keyword and string-similarity grading, and the literature of Chapter 2.
- **Proposed method:** the seven-layer architecture, the scoring model of Equations (1) to (4) and the review routing rules of Table 4.1.
- **Experiments:** latency of OCR, retrieval and evaluation, agreement with faculty markers (Pearson r = 0.94) and the verification of low-confidence cases.
- **Discussion and conclusion:** limitations, the role of the teacher, and future work on diagrams, regional languages and offline operation.

B.4 SUPPORTING DOCUMENTS

The acceptance e-mail or letter, the certificate of presentation and the first page of each published paper are to be attached after this page.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3994.

[3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[4] G. Kortemeyer, “Could an AI answer science examinations? Performance of GPT-4 on introductory physics exams,” Phys. Rev. Phys. Educ. Res., vol. 19, no. 1, p. 010132, 2023.

[5] C. Grévisse et al., “Automated short-answer grading using Large Language Models in medical education,” BMC Med. Educ., vol. 24, no. 1, p. 112, 2024.

[6] M. Henkel et al., “Evaluating Large Language Models for automated short answer grading in primary and secondary education,” Comput. Educ. Artif. Intell., vol. 5, p. 100185, 2023.

[7] K. Pecuchova et al., “Generative AI for open-ended assessment and feedback in higher education,” Educ. Inf. Technol., vol. 29, pp. 5421–5445, 2024.

[8] Y. Xu and J. Corter, “Graph-augmented Retrieval-Augmented Generation for short answer grading,” in Proc. Int. Conf. Artif. Intell. Educ. (AIED), 2024, pp. 310–322.

[9] Elsevier Procedia, “Interactive feedback loops and automated scoring pipelines in computer science education,” Procedia Comput. Sci., vol. 235, pp. 1204–1215, 2024.

[10] Y. Du et al., “PaddleOCR: An ultra-lightweight OCR system,” arXiv preprint arXiv:2009.09941, 2020.

[11] Qdrant Team, “Qdrant: Vector Database for the Next Generation of AI Applications,” Technical Documentation, 2024. [Online]. Available: https://qdrant.tech/documentation/

[12] A. Vaswani et al., “Attention Is All You Need,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[13] T. B. Brown et al., “Language Models are Few-Shot Learners,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[14] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 5776–5788.

[15] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.

[16] S. Burrows, I. Gurevych, and B. Stein, “The Eras and Trends of Automatic Short Answer Grading,” Int. J. Artif. Intell. Educ., vol. 25, no. 1, pp. 60–117, 2015.

[17] M. Mohler, R. Bunescu, and R. Mihalcea, “Learning to Grade Short Answer Questions using Semantic Similarity Measures and Dependency Graph Alignments,” in Proc. ACL-HLT, 2011, pp. 752–762.

[18] D. Ramesh and S. K. Sanampudi, “An automated essay scoring systems: a systematic literature review,” Artif. Intell. Rev., vol. 55, no. 3, pp. 2495–2527, 2022.

[19] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track, vol. 36, 2023.

[20] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 35, 2022, pp. 24824–24837.